

Contents

guide 01 distserve: Serving Graduation Lens for DistServe	1
0. 为什么毕业专题还要再读 DistServe	2
1. 毕业作品的核心转变	2
2. DistServe 论文内容快速但完整地重建	3
2.1 两阶段 LLM serving	3
2.2 Colocated serving 的结构性干扰	3
2.3 为什么 chunked prefill 没彻底解决	3
2.4 DistServe 的基本方案	4
2.5 Goodput 目标	4
3. 论文里的数学: 你要带到毕业项目里的部分	5
3.1 M/D/1 排队直觉	5
3.2 Parallelism 不是越多越好	5
3.3 KV transfer 变成 portfolio 里的 hidden cost	6
4. 论文实验证据链	6
4.1 Figure 1 的教学价值	6
4.2 实验平台和任务	7
4.3 Baselines	7
4.4 主结果	8
4.5 Latency breakdown 和 simulator accuracy	8
5. 把 DistServe 翻译成毕业验收模型	9
5.1 从 request_passes 到 good_answer	9
5.2 从 per-GPU goodput 到 cost_per_good_request	9
5.3 从 placement search 到 model routing	9
6. 本仓库代码连接	10
6.1 Capstone 主线	10
6.2 新增 scorecard	11
6.3 单元测试对应的学习动作	11
7. 你应该如何复现实验	12
8. 毕业项目的报告标准	12
9. AI agent 应该怎么帮你学	13
9.1 Reviewer	13
9.2 Examiner	14
9.3 Experiment Coach	14
10. 闭卷自测	14
11. 最后一页: 你真正要带走的能力	14

guide_01_distserve: Serving Graduation Lens for DistServe

原论文: DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

本地原文 PDF: [learning/serving-graduation/paper/01_distserve.pdf](#)

作者: Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, Hao Zhang

机构: Peking University, StepFun, UC San Diego

版本: arXiv 2401.09670v3, 2024-06-06

本专题定位: serving graduation capstone, 把论文思想转成可验收的端到端 serving 作品

0. 为什么毕业专题还要再读 DistServe

在 distributed-inference 专题里, DistServe 是一篇分布式推理论文: 它讲 prefill 和 decode 为什么互相干扰, 为什么要分离资源池, 为什么要用 goodput 而不是总吞吐。

在 serving-graduation 里, 同一篇论文要换一个读法: 你要把 DistServe 当成毕业作品的验收标准。真正的毕业项目不是“能跑一个 demo”, 而是能说明:

- 我的 workload 是什么。
- 我的 SLO 是什么。
- 我的请求什么时候算成功。
- 我的延迟、正确性和成本如何一起评估。
- 我的 routing, cache, thinking budget, model tier 如何影响 goodput。
- 我的 report 能不能支撑一个部署决策。

所以这一篇 guide 不只是重复论文机制。它要回答:

如何把 DistServe 的 per-GPU goodput 思维,
迁移到本仓库的 serving graduation capstone?

一句话版本:

DistServe 教你不要只问系统能吐多少 token,
而要问在给定 SLO 和成本下, 多少请求真的达标。

serving graduation 则要求你把这个问题的扩展到正确性、模型路由、reasoning budget 和端到端报告。

1. 毕业作品的核心转变

前面的专题很多是在学单个技术点:

- LoRA: 怎么低成本适配。
- DPO/RLHF: 怎么让输出更符合偏好。
- RAG/agent/tool: 怎么接外部上下文和动作。
- vLLM/SGLang/spec decoding: 怎么提升推理效率。
- distributed inference: 怎么处理 GPU 并行、KV transfer、prefill/decode 分离。

毕业专题要把这些线接起来。你不能只说“我的 R1-Zero 输出更长”, 也不能只说“我的小模型更便宜”。你要能做一个工程判断:

在这个应用里, 哪个 checkpoint 或 routing 策略最值得部署?

这个问题至少有四个维度:

- correctness: 答案是否对。
- latency: 用户是否等得起。
- cost: 每个好请求花多少钱。
- operability: 失败、冷启动、缓存、路由是否可控。

DistServe 的价值是把 serving 从“速度秀”拉回到 SLO 和 goodput。毕业专题要进一步把它拉到:

good_request = correct and within_latency_slo and within_cost_budget

2. DistServe 论文内容快速但完整地重建

2.1 两阶段 LLM serving

LLM 在线生成可以拆成两个阶段。

prefill:

```
input prompt tokens: [x1, x2, ..., xL]
model processes all prompt tokens in one forward pass
first output token is produced
KV cache for the prompt is materialized
```

decode:

```
model generates one token at a time
each step reads previous KV cache
new KV is appended
tokens stream back to the user
```

用户感知上:

```
TTFT = time to first token
TPOT = time per output token
request_latency = TTFT + TPOT * output_tokens
```

DistServe 认为在线服务至少要同时看 TTFT 和 TPOT。一个系统如果第一口响应慢, 聊天体验会差; 如果第一口很快但后续 token 卡顿, 用户仍然觉得慢。

2.2 Colocated serving 的结构性干扰

传统 LLM serving 系统通常把 prefill 和 decode 放在同一组 GPU 上。原因很自然:

- 两阶段共享模型权重。
- decode 需要 prefill 产生的 KV cache。
- continuous batching 可以把不同请求混在一起提高 GPU 利用率。

问题也随之出现:

```
new long prompt
-> long prefill job
-> active decode requests wait
-> TPOT gets worse

many active decode requests
-> new prefill shares batch and queue
-> TTFT gets worse
```

论文 Figure 2 通过 13B 模型 profiling 说明: 在 decoding-only batch 里加一个 prefill job 会显著拉长 batch execution time, prompt 越长越明显。反过来, decode jobs 也会拖慢 prefill。

这不是单个 scheduler bug, 而是两个阶段在同一个资源池里互相抢时间。

2.3 为什么 chunked prefill 没彻底解决

chunked prefill 把长 prompt 切成多个 chunk, 让 decode step 可以插入执行。它能缓解长 prefill 一次性阻塞, 但论文指出:

- 它仍然让 prefill 和 decode 共享 GPU。
- 它是在 TTFT 和 TPOT 之间做折中。
- prompt 很长时, 后续 chunk 需要反复读取之前的 KV cache。
- 对长上下文, 额外内存访问会更明显。

所以 chunked prefill 是一种缓解, 不是解除耦合。

2.4 DistServe 的基本方案

DistServe 把两阶段拆开:

```

request
|
v
controller
|
v
prefill instance
|
| sends KV cache
v
decode instance
|
v
stream output

```

两个 instance 都可以有自己的模型副本、GPU 数、parallelism 配置和 batching 策略。收益:

- prefill pool 优化 TTFT。
- decode pool 优化 TPOT。
- 两阶段不再直接互相阻塞。
- resource allocation 可以按 workload 和 SLO 自动搜索。

代价:

- 需要传 KV cache。
- 需要更多 orchestration。
- 可能需要额外模型副本。
- placement 必须考虑带宽。

2.5 Goodput 目标

DistServe 不把 total tokens/s 当作核心目标。它定义的是 per-GPU goodput:

```

request_passes =
    (TTFT <= TTFT_SLO) and (TPOT <= TPOT_SLO)

attainment =
    passed_requests / total_requests

goodput =
    max request_rate that reaches attainment target

```

```
per_gpu_goodput =  
    goodput / number_of_GPUs
```

这句话是毕业专题的关键: 指标必须描述“ 真实达标的请求”, 不能只描述系统内部忙不忙。

3. 论文里的数学: 你要带到毕业项目里的部分

3.1 M/D/1 排队直觉

论文用 prefill-only instance 解释不同 parallelism 的选择。假设:

- 请求到达率是 R 。
- 每个请求执行时间是 D 。
- 利用率条件是 $R * D < 1$ 。

平均 TTFT:

$$\text{Avg_TTFT} = D + R * D^2 / (2 * (1 - R * D))$$

第一项是执行时间, 第二项是排队时间。它告诉你:

- rate 低时, 单请求执行时间主导。
- rate 高时, 排队时间可能爆炸。
- serving 系统不能只测一条 request 的 latency。

毕业 capstone 也一样。你不能只跑一次 Janet 鸭蛋题就宣布某个 checkpoint 最好。你至少要说明在目标请求率下, 哪些请求还满足 SLO。

3.2 Parallelism 不是越多越好

prefill 和 decode 的 parallelism 偏好不同:

- prefill 长 prompt 时更 compute-heavy, 可能需要 intra-op parallelism 降低 TTFT。
- decode 单步更 memory-bandwidth sensitive, 需要足够 batch 或 replica 才能降低 TPOT。
- rate, SLO, prompt length, output length 都会改变最优资源分配。

毕业项目里的对应关系:

```
small model:  
    lower latency and cost  
    maybe lower correctness
```

```
large model:  
    higher capability  
    higher latency and cost
```

```
thinking model:  
    stronger reasoning trace  
    higher output length and latency
```

```
router:  
    sends easy requests to cheap model  
    sends hard requests to expensive model
```

这正是 multi_model_router.py 和 serving_scorecard.py 要一起表达的东西。

3.3 KV transfer 变成 portfolio 里的 hidden cost

DistServe 拆开 prefill/decode 后, 必须传 KV cache:

```
kv_bytes_per_token =  
    2 * num_layers * num_kv_heads * head_dim * bytes_per_value  
  
kv_payload_bytes =  
    prompt_tokens * kv_bytes_per_token  
  
transfer_ms =  
    kv_payload_bytes * 8 / bandwidth_bits_per_second * 1000
```

论文给的例子是: OPT-66B 的 512-token request, KV cache 大约 1.13GB。10 rps 时, 每秒要传大约 11.3GB, 也就是约 90Gbps 才能让传输不显眼。

毕业项目里, 这个思想要迁移成:

- reasoning trace 不是免费, 输出越长 TPOT 和 cost 越大。
- RAG 检索不是免费, 会引入 IO 和 latency。
- 多模型路由不是免费, router 误判会影响 correctness 或 cost。
- embedding/VLM 服务不是免费, 会多一个 encoder 和数据传输路径。
- cache 命中不是免费, 需要正确路由和状态管理。

DistServe 让你习惯问: “我引入的新模块在系统路径上增加了什么 hidden cost?”

4. 论文实验证据链

4.1 Figure 1 的教学价值

论文用一个 13B LLM synthetic workload 做开场:

- input length = 512。
- output length = 64。
- 单张 NVIDIA A100 80GB。
- 90% SLO attainment。

结果直觉:

colocated existing system:
about 1.6 rps per GPU

prefill-only:
about 5.6 rps

decode-only:
about 10 rps

ideal split:
2 GPUs for prefill, 1 GPU for decode
about 10 rps overall
about 3.3 rps per GPU
about 2.1x colocated per-GPU goodput

这张图的毕业意义: 一个系统看似慢, 可能不是模型本身慢, 而是资源池耦合导致两个阶段互相阻塞。capstone 里也一样, 某个 checkpoint 表现差, 你要拆开看:

- 是模型能力差。
- 是 latency 超 SLO。
- 是 output 太长。
- 是 router 选错。
- 是成本太高。
- 是 report 指标没定义好。

4.2 实验平台和任务

DistServe 实验:

- 4 个节点。
- 每节点 8 张 NVIDIA SXM A100 80GB。
- 节点内 NVLINK。
- 跨节点 25Gbps。
- 大部分实验用 low node-affinity placement, 因为跨节点带宽有限。

模型:

- OPT-13B, 26GB。
- OPT-66B, 132GB。
- OPT-175B, 350GB。

任务:

- Chatbot: ShareGPT。
- Code completion: HumanEval。
- Summarization: LongBench。

SLO 形态:

Chatbot:

TTFT matters for responsiveness
TPOT matters for smooth streaming

Code completion:

TTFT is very tight because user waits inside editor

Summarization:

input is long
TTFT can be looser
TPOT matters because output should finish quickly

毕业 capstone 也应该这样定义 workload。不要只说“我测了一道题”; 要说这道题代表什么用户场景, 它的 SLO 是什么。

4.3 Baselines

论文 baseline:

vLLM:

- continuous batching。
- PagedAttention。
- colocated prefill/decode。

DeepSpeed-MII:

- chunked prefill。
- 能缓解长 prefill 干扰。
- 仍然不是完全 disaggregation。

这给毕业项目一个标准: baseline 必须有意义。你的 capstone 不能只展示最强模型, 还要展示:

- vanilla baseline。
- LoRA 后变化。
- DPO 后变化。
- R1-style reasoning 后变化。
- small but clean model 的替代方案。

本仓库的 `graduation_e2e/ckpts.py` 正是这个结构。

4.4 主结果

论文总结果:

DistServe can serve up to 7.4x more requests
or support 12.6x tighter SLO,
while keeping more than 90% requests within latency constraints.

按任务看:

- Chatbot on ShareGPT: DistServe 对 vLLM 可承载约 2.0x 到 4.6x 更高 request rate。
- Code completion on HumanEval: 对 vLLM 有 5.7x 更高 request rate, 1.4x 更紧 SLO。
- Summarization on LongBench: 对 vLLM 有 4.3x 更高 request rate, 12.6x 更紧 SLO。

毕业专题读这些结果时, 不要只背倍数。你要问:

- 每个倍数对应什么 SLO。
- 哪个 baseline 被打败。
- 是 rate 提升还是 SLO 变紧。
- 是哪个 workload 形态带来的收益最大。
- 如果换成自己的 capstone, 哪个模型或路由策略对应这个提升。

4.5 Latency breakdown 和 simulator accuracy

论文 Figure 10 看 OPT-175B on ShareGPT 的 latency breakdown:

- KV transmission 在总 latency 中小于 0.1%。
- 超过 95% 请求的 transmission time 小于 30ms。
- 原因是 placement algorithm 把相同 stage 放在同一 node 上, 利用 NVLINK。

Table 2 看 simulator accuracy:

- simulator 和 real system 的 SLO attainment 误差小于 2%。

毕业项目里的对应标准:

- 如果你的 report 用 mock cost 或 mock latency, 要明确它是 proxy。

- 如果你的 router 用 heuristic, 要给出失败场景。
- 如果你的 scorecard 说某模型最好, 要能解释打分函数。
- 如果没有真实压测, 要承认这是教学模拟, 不要包装成生产指标。

5. 把 DistServe 翻译成毕业验收模型

5.1 从 request_passes 到 good_answer

DistServe:

```
request_passes =
  TTFT <= SLO_TTFT
  and TPOT <= SLO_TPOT
```

毕业 capstone:

```
answer_passes =
  correct == True
  and latency <= latency_slo
  and per_token_latency <= tpot_slo
  and cost <= cost_budget
```

这里 correctness 是毕业作品必须加上的门槛。一个很快但答错的模型不能算 goodput。一个很准但每次都太慢的 reasoning model 也不能无条件胜出。

5.2 从 per-GPU goodput 到 cost_per_good_request

DistServe 关注:

```
per_gpu_goodput = goodput / number_of_GPUs
```

毕业项目可以用:

```
attainment = good_answers / total_answers
effective_goodput = request_rate * attainment
cost_per_good_request = total_cost / good_answers
```

这个转换让你能比较:

- LoRA: 便宜且答对, latency 低。
- DPO: 稍慢但推理更清晰。
- R1-Zero: reasoning trace 强, 但 latency 高。
- Phi-tiny: 小模型可能在特定任务上表现清洁。
- vanilla: 快但错, 不能算 good answer。

5.3 从 placement search 到 model routing

DistServe placement search 的目标:

```
given workload and SLO,
choose prefill/decode resource split
```

毕业项目里对应:

```
given query complexity and SLO,
choose model tier or checkpoint
```

比如:

```
easy query:
  route to small model

medium query:
  route to LoRA or DPO model

hard reasoning query:
  route to thinking model

strict latency query:
  avoid long thinking trace
```

这就是 `multi_model_router.py` 的教学意义。它现在只是 heuristic, 但它让你看到: routing 不是“挑最强模型”, 而是在 capability, latency, cost 之间找一个满足 SLO 的选择。

6. 本仓库代码连接

6.1 Capstone 主线

关键文件:

- `learning/serving-graduation/src/graduation_e2e/ckpts.py`
- `learning/serving-graduation/src/graduation_e2e/compare.py`
- `learning/serving-graduation/src/graduation_e2e/report.py`
- `learning/serving-graduation/src/multi_model_router.py`
- `learning/serving-graduation/src/serving_scorecard.py`
- `learning/serving-graduation/src/tests/test_graduation.py`

`ckpts.py` 里有 5 个 mock checkpoint:

```
vanilla:
  fast
  wrong

lora:
  fast
  correct
  brief reasoning

dpo:
  a bit slower
  correct
  explicit step-by-step

r1_zero:
  slowest
  correct
  explicit think trace

phi_tiny:
  mid latency
```

```
correct
clean small-model answer
```

这个结构刚好对应毕业作品的五线对比。

6.2 新增 scorecard

serving_scorecard.py 把 DistServe 的 goodput 思维接到 capstone:

```
from graduation_e2e.compare import run_compare
from serving_scorecard import GraduationSLO, score_report, effective_goodput, rank_candidates

report = run_compare()
slo = GraduationSLO(max_ttft_ms=70, max_tpot_ms=8)

scores = score_report(report, slo)
summary = effective_goodput(scores, request_rate_rps=2.0)
ranked = rank_candidates(scores)

print(summary)
print(ranked[0])
```

含义:

- score_report: 给每个 checkpoint 计算是否达标。
- GraduationSLO: 定义 latency 和 per-token latency 门槛。
- effective_goodput: 用 request rate 乘以 attainment。
- rank_candidates: 先选通过 SLO 的候选, 再按成本和延迟排序。

这不是生产级成本模型, 但它是毕业项目必须具备的最小形状:

```
I can define a good request.
I can measure attainment.
I can explain why one candidate beats another.
I can name what is still mocked.
```

6.3 单元测试对应的学习动作

新增测试里有三件事:

```
wrong or slow outputs are not good:
  vanilla fails correctness
  r1_zero fails latency under strict SLO

goodput is request_rate times attainment:
  3 passing candidates out of 5
  request_rate = 2.0
  goodput = 1.2 rps

ranker picks cheapest passing candidate:
  lora wins under the chosen strict SLO
```

这三个测试就是 DistServe 思想的毕业版缩影: 好系统不是最慢最强, 也不是最快最错, 而是在当前 SLO 下产出最多合格请求。

7. 你应该如何复现实验

先跑环境:

```
.venv\Scripts\python.exe learning\serving-graduation\environment\verify_env.py
```

再跑测试:

```
.venv\Scripts\python.exe -m pytest learning\serving-graduation\src\tests -q
```

然后生成 capstone report:

```
.venv\Scripts\python.exe -m graduation_e2e.run --out report
```

如果你在 repo root 直接运行 module, 需要把 learning/serving-graduation/src 加到 PYTHONPATH 或用测试里的 sys.path 方式。更适合的学习动作是在 Python 片段里显式导入:

```
import sys
```

```
sys.path.insert(0, "learning/serving-graduation/src")
```

```
from graduation_e2e.compare import run_compare
```

```
from serving_scorecard import GraduationSLO, effective_goodput, score_report
```

```
scores = score_report(run_compare(), GraduationSLO(70, 8))
```

```
print(scores)
```

```
print(effective_goodput(scores, 2.0))
```

你要观察:

- SLO 放宽时, R1-Zero 是否通过。
- SLO 收紧时, DPO 或 Phi-tiny 是否失败。
- 如果不要正确ness, vanilla 是否会错误地变成“好候选”。
- 如果 request_rate 提高, effective_goodput 如何变化。

8. 毕业项目的报告标准

DistServe 的论文报告结构很值得学:

problem:

colocated prefill/decode causes interference

method:

disaggregate phases

optimize placement

metric:

SLO attainment and per-GPU goodput

evidence:

multiple models

multiple workloads

baselines

ablation

latency breakdown

simulator validation

limitations:

- resource-constrained setting
- throughput-only setting
- fault tolerance
- network assumptions

你的 graduation report 也应该有同样骨架:

problem:

- which checkpoint or serving strategy should we deploy?

method:

- compare five model paths
- add routing and scorecard

metric:

- correctness
- latency
- per-token latency
- effective goodput
- cost per good request

evidence:

- side-by-side responses
- latency and cost proxy
- SLO pass/fail
- ranking

limitations:

- mock ckpts
- one canonical question
- no real load test
- proxy cost model

9. AI agent 应该怎么帮你学

不要让 agent 直接替你写总结。你要让 agent 做三个角色:

9.1 Reviewer

给 agent 这个 prompt:

请你 review 我的 serving graduation report。

不要改写文案，只找指标漏洞。

请检查我是否定义了 good request, SLO, attainment, cost proxy, baseline, limitation。
每次只指出 3 个最严重问题，并要求我用代码文件名回答。

9.2 Examiner

请你用 DistServe 的视角考我 serving graduation。
一次只问一个问题。
问题必须要求我同时引用论文机制和本仓库代码。
如果我回答只讲论文或只讲代码，请判为不合格。

9.3 Experiment Coach

请你给我 3 个 SLO 设置：
一个偏严格 latency，一个偏严格 correctness，一个偏严格 cost。
让我预测 lora, dpo, r1_zero, phi_tiny 谁会胜出。
然后要求我运行 serving_scorecard.py 验证。
正确用 agent 的关键：让它逼你回到代码和指标，不让它用漂亮段落把你哄过去。

10. 闭卷自测

你读完应该能回答：

- 为什么 DistServe 的 goodput 指标比 total tokens/s 更适合在线服务。
- 为什么 serving graduation 还要在 goodput 之外加 correctness。
- 为什么 vanilla 快但错，不能算 good request。
- 为什么 R1-Zero 推理强，但在严格 latency SLO 下可能不是最佳部署候选。
- multi_model_router.py 和 DistServe placement search 的类比是什么。
- serving_scorecard.py 里哪一行体现了 attainment。
- 如果把 max_ttft_ms 从 70 放宽到 100，你预测哪个 checkpoint 会通过。
- 如果把 require_correct 改成 False，指标会出现什么坏味道。
- 为什么 mock cost model 必须在 report 里声明局限。
- 如果要从毕业 toy 升级到真实服务，最缺哪三类数据。

11. 最后一页：你真正要带走的能力

DistServe 的毕业意义不是“我知道 prefill/decode disaggregation”。

真正要带走的是一种工程判断顺序：

1. define workload
2. define SLO
3. define good request
4. measure attainment
5. compute effective goodput
6. compare baselines
7. explain cost and hidden costs
8. name limitations
9. use code to verify the story

当你能用这个顺序解释本仓库的 5-checkpoint capstone，你才真正从“会跑 demo”进入“会做 serving 决策”。